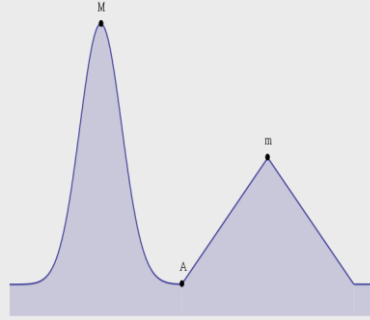
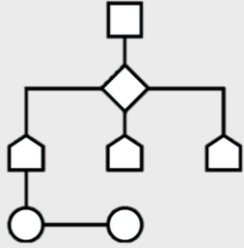


UNE APPROCHE DE L'OPTIMISATION

Les Algorithmes GLOUTONS

Les Problèmes d'Optimisation



Définition & Stratégie de Résolution

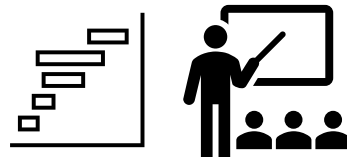
Les Problèmes d'Optimisation

- **Contexte :**
 - De nombreux problèmes comportent plusieurs solutions.
 - Cependant toutes les solutions ne se valent pas et on peut appliquer différents critères de performances, c'est l'optimisation.
 - Optimiser un problème, c'est mener un démarche contribuant à proposer une configuration (choix de paramètres) qui vont conduire à résoudre le problème tout en minimisant ou maximisant une ou plusieurs contraintes énoncées.

Les Problèmes d'Optimisation

- **Exemples :**

- Dans une tournée de livraison, déterminer le parcours minimisant la distance et/ou le temps tout en assurant l'ensemble des livraisons.
- Optimisation d'un emploi de temps (scolaire par exemple) : l'ensemble des activités (cours par exemple) devront être placées en minimisant les trous et tout en assurant la disponibilité des ressources (salle devant être libre, professeur disponible).
- Minimiser la masse d'une pièce mécanique dans une automobile.
- Trouver la disposition une disposition de colis/cartons dans un camion pour minimiser le volume occupé.



Les Problèmes d'Optimisation

- **Catégories :**
 - Il existe donc plusieurs catégories/types de problèmes.
 - Il n'existe pas de méthodes et/ou d'algorithmes universels.
 - Par ailleurs, un problème peut également ne pas avoir de solution !

Présentation des Algorithmes Gloutons

Principe des Algorithmes Gloutons

- **Objectif :**
 - Les algorithmes gloutons sont une catégorie d'algorithmes proposant une solution au problème posé avec le souci de fournir une solution optimisée mais sans garantir que la solution proposée est la plus optimale de toutes.
 - En contre partie de la performance absolue, les algorithmes gloutons sont relativement simple à mettre en œuvre.

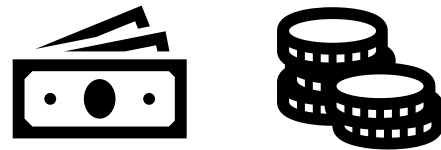
Principe des Algorithmes Gloutons

- Principe :
 - Les algorithmes gloutons s'appliquent à des problèmes qui peuvent se résoudre par étape.
 - Un algorithme glouton consiste à **choisir la solution la plus optimale pour chaque étape** de résolution.
 - On espère ainsi d'aboutir sur une solution optimale globalement.
 - Cependant la pratique montre (on le verra) que la somme de solutions optimales locales n'aboutie pas systématiquement à la solution globale la plus optimale.

Principe des Algorithmes Gloutons

- Définition d'un algorithme glouton :
 - **Stratégie algorithmique de résolution d'un problème décomposé en sous-problèmes/étapes. Pour chaque étape/sous-problèmes la solution locale la plus optimale est systématiquement choisie sans considérer la performance globale du problème.**
- Remarque :
 - Dans un algorithme glouton on ne remet jamais en cause le choix d'une solution locale précédemment adoptée.
 - Il existe d'autres algorithmes, notamment des algorithmes dits dynamiques qui fonctionnent sur une approche comparative des solutions locales afin de choisir la configuration de solutions locales qui permet d'aboutir sur une solution globale la plus performante.

Applications



Le rendu de monnaie

Le Rendu de Monnaie

- **Exposé du problème à résoudre :**

- Restituer une somme avec tout ou partie des pièces et monnaies proposée par le système monétaire.

- **Contrainte d'optimisation :**

- La somme due doit être restituée avec le nombre **minimal** de pièces.

Le Rendu de Monnaie

- **Exemple :**

- Pour donner/rendre une somme $S = 3,88 \text{ €}$.
- Si nous disposons de l'ensemble des pièces disponibles.
- On peut rendre la monnaies suivant les solutions suivantes :
 - $3,88\text{€} = 1 \times 2\text{€} + 1 \times 1\text{€} + 1 \times 0,50\text{€} + 1 \times 0,20\text{€} + 1 \times 0,10\text{€} + 1 \times 0,05\text{€} + 1 \times 0,02\text{€} + 1 \times 0,01\text{€}$
 - Soit un total de 8 pièces.



Le Rendu de Monnaie

- **Exemple :**
 - Mais on aurait pu à l'opposer proposer :
 - $3,88\text{€} = 388 \times 0,01\text{€}$
 - Soit un total de 388 pièces !!!!



Le Rendu de Monnaie

- **Exemple :**
 - Ou d'autre combinaisons :
 - $3,88\text{€} = 3 \times 1\text{€} + 4 \times 0,20\text{€} + 1 \times 0,05\text{€} + 3 \times 0,01\text{€}$
 - Pour un total de 11 pièces.



Le Rendu de Monnaie

- **L'algorithme glouton de rendu de monnaie :**

- La 1^{ère} proposition correspond au résultat de l'algorithme glouton de rendu de monnaie :
- $3,88\text{€} = 1 \times 2\text{€} + 1 \times 1\text{€} + 1 \times 0,50\text{€} + 1 \times 0,20\text{€} + 1 \times 0,10\text{€} + 1 \times 0,05\text{€} + 1 \times 0,02\text{€} + 1 \times 0,01\text{€}$
- En effet on a cherché d'abord la pièce/billet de plus grande valeur, soit ici la pièce de 2€
- Il reste alors $3,88 - 2 = 1,88\text{€}$, la plus grande valeur de pièce/billet sur ce reste est la pièce de 1€
- Il reste alors $1,88 - 1 = 0,88\text{€}$, la plus grande valeur de pièce/billet sur ce reste est la pièce de 0,50€
- Il reste alors $0,88 - 0,50 = 0,38\text{€}$, la plus grande valeur de pièce/billet sur ce reste est la pièce de 0,20€
- Il reste alors $0,38 - 0,20 = 0,18\text{€}$, la plus grande valeur de pièce/billet sur ce reste est la pièce de 0,10€
- Il reste alors $0,18 - 0,10 = 0,08\text{€}$, la plus grande valeur de pièce/billet sur ce reste est la pièce de 0,05€
- Il reste alors $0,08 - 0,05 = 0,03\text{€}$, la plus grande valeur de pièce/billet sur ce reste est la pièce de 0,02€
- Il reste alors $0,03 - 0,02 = 0,01\text{€}$, la plus grande valeur de pièce/billet sur ce reste est la pièce de 0,01€

Le Rendu de Monnaie

- Analyse, critiques et perspectives :

- Le système de monnaie européenne est dit **canonique**.
- Un **système canonique** est un système : Qui a toujours **une solution** ET l'application d'un algorithme glouton sur ce type de système permet de toujours obtenir une solution **optimale**.



Le Rendu de Monnaie

- Contre exemple :

- On désire boire une boisson chaude au prix de **0,87 €**
- On dispose d'une pièce de **1 €**
- Mais la machine ne dispose plus de pièces **1 centime !**
- Or pour rendre la monnaie de **0,13 €**, il nous selon un algorithme glouton :
 - $1 \times 0,10\text{€} + 1 \times 0,02\text{€} + 1 \times 0,01\text{€}$
- Dans cette configuration particulière, le système monétaire **perd son caractère canonique**, l'algorithme glouton n'arrive pas à trouver une solution exacte !
- La solution exacte peut ou ne peut pas exister dans un système **non canonique**...
- Cependant ici il existait bien une solution exacte !, la solution est la suivante :
 - $1 \times 0,05\text{€} + 4 \times 0,02\text{€} = 0,13\text{€}$



Le Rendu de Monnaie

- Implémentation de l'algorithme glouton de rendu de monnaie :

- Notre programme utilisera 2 listes :

- 1 liste désignée ***Sn*** qui contient dans l'ordre décroissant et en centimes, les pièces et/ou billets disponibles dans notre système monétaire.

- Exemple :

$S_n = [200, 100, 50, 20, 10, 5, 2, 1]$



Le Rendu de Monnaie

- Implémentation de l'algorithme glouton de rendu de monnaie :

- Pourquoi exprimer le système monétaire en centimes ?

- La stratégie est ici de manipuler que des entiers et non des flottants.
 - En effet la manipulation de flottants nous confronte au problème d'arrondi de certaines valeurs (conséquence de la conversion en binaire des valeurs décimales)

- Exemples :

```
>>> 0.1+0.2
0.30000000000000004

>>> 0.3-0.1
0.19999999999999998
```



Le Rendu de Monnaie

- Implémentation de l'algorithme glouton de rendu de monnaie :

- Le résultat sera une liste nommée **rendu** et qui contient le nombre de pièces/et ou billets selon l'ordre de la liste **Sn**

$$\text{rendu} = [3, 1, 1, 2, 0, 0, 1, 1]$$

$$S_n = [200, 100, 50, 20, 10, 5, 2, 1]$$

$$\text{valeur rendu} = 3 \times \frac{200}{100} \text{€} + 1 \times \frac{100}{100} + 1 \times \frac{50}{100} + 2 \times \frac{20}{100} + 0 + 0 + 1 \times \frac{2}{100} + 1 \times \frac{1}{100} = 7,93\text{€}$$

Le Rendu de Monnaie

- Implémentation de l'algorithme glouton de rendu de monnaie :
 - Réaliser l'implémentation sous Python du de l'algorithme glouton de rendu de monnaie.
 - L'algorithme suit la description suivante :
 - **Tant que la somme à rendre n'est pas nulle recommencer les étapes suivantes :**
 - **Choisir la pièce de plus grande valeur du système monétaire mais inférieure à la somme à rendre**
 - **Retirer de la somme à rendre la valeur de pièce précédemment choisie et reprendre à l'étape précédente**

Le Rendu de Monnaie

- **Conseils et aide :**
 - Vous aurez besoin de 2 boucles pour réaliser votre programme :
 - 1 boucle de 1^{er} niveau qui poursuit l'algorithme tant que qu'il reste de l'argent à rendre;
 - 1 boucle de 2^e niveau qui permet de sélectionner la valeur correcte.

Le Rendu de Monnaie

- **Éléments et trame du programme :**

- On vous conseille de procéder par fonction.
- Vous pouvez créer une 1^{er} fonction qui assure la conversion de la liste Sn en centimes d'euros par exemple :

```
2 def convMonnaie(Sn, facConv):  
3     for i in range(len(Sn)):  
4         Sn[i] = Sn[i] * facConv  
5
```

Le Rendu de Monnaie

- Éléments et trame du programme :

- La 2^e fonction assure le rendu monnaie et utilise la fonction de conversion pour travailler avec des centimes :

```
5
6 def rendreMonnaie(s, Sn):
7     # Conversion des valeurs en centimes (Ceci permet d'éviter les erreurs d'arrondi
8     convMonnaie(Sn, 100)
9     s = s*100
10
11     rendu = [0] * len(Sn)
12     reste = s
13     Sn.sort(reverse = True) # On tri le système de monnaie dans l'ordre décroissant
14     i = 0
15     while Code à compléter :
16         while Code à compléter
17             i = i + 1
18
19             reste = reste - Sn[i]
20             rendu[i] = rendu[i] + 1
21
22     convMonnaie(Sn, 1/100) # Reconversion en euros.
23     return rendu
24
```

Le Rendu de Monnaie

- **Éléments et trame du programme :**

- Enfin, on veut demande d'implémenter une dernière fonction qui permet de présenter le résultat :

```
26 def detailRendu(rendu, Sn):  
27     print("\nLe rendu est constitué :\n"  
28           "-----\n")  
29     for i in range(len(rendu)):  
30         print(str(rendu[i]) + "\tpièce(s)/billet(s) de :\t" + str(Sn[i]))  
31
```

Le Rendu de Monnaie

- **Éléments et trame du programme :**

- Le programme principal assure la coordination de l'ensemble :

```
33 # Exemple :  
34 # -----  
35  
36 Sn = [0.01, 0.02, 0.05, 0.10, 0.20, 0.50, 1, 2]  
37 somme = 7.93  
38 print("rendu = " + str(rendreMonnaie(somme, Sn)))  
39 rendu = rendreMonnaie(somme, Sn)  
40 detailRendu(rendu, Sn)  
41
```

Le Rendu de Monnaie

- **Résultat :**
 - Votre programme devrait alors présenter ce type de résultat :

```
>>> %Run '01-Rendu de Monnaie(élève).py'
rendu = [3, 1, 1, 2, 0, 0, 1, 1]

Le rendu est constitué :
-----

3      pièce(s)/billet(s) de : 2.0
1      pièce(s)/billet(s) de : 1.0
1      pièce(s)/billet(s) de : 0.5
2      pièce(s)/billet(s) de : 0.2
0      pièce(s)/billet(s) de : 0.1
0      pièce(s)/billet(s) de : 0.05
1      pièce(s)/billet(s) de : 0.02
1      pièce(s)/billet(s) de : 0.01
>>>
```

Applications



Le Problème du Sac à Dos

Le Problème du Sac à Dos

- **Description du problème :**

- Selon votre morale, vous êtes un cambrioleur ou un explorateur qui doit emporter dans son unique sac à dos des objets ayant 2 propriétés remarquables :
 - Chaque objet à une valeur marchande connue
 - Mais aussi chaque objet à une masse qui est connue

- **Objectif :**

- Emporter la plus grande valeur d'objets (somme des valeurs de chaque objet) sachant que votre sac à dos à une charge maximale.

Le Problème du Sac à Dos

- Exemple :

- Les objets suivants sont disponibles en quantité suffisante pour permettre toutes les combinaisons imaginables.

Identifiant objet	Obj1	Obj2	Obj3	Obj4
Masse (kg)	18 kg	9 kg	6 kg	3 kg
Prix (€)	60 €	28 €	16 €	7 €

- Le sac à dos a une capacité de charge **maximale de 30 kg**.



Le Problème du Sac à Dos

- **Résolution :**

- Nous avons ici un problème d'optimisation encore. Nous allons le résoudre avec un algorithme glouton.
- Pour ce faire nous avons besoin de connaître le prix massique de chaque objet, ce qui donne le nouveau tableau ci-dessous :

Identifiant objet	Obj1	Obj2	Obj3	Obj4
Masse (kg)	18 kg	9 kg	6 kg	3 kg
Prix (€)	60 €	28 €	16 €	7 €
Prix massique (€/kg)	3,33 €/kg	3,11 €/kg	2,67 €/Kg	2,33 €/kg

Le Problème du Sac à Dos

- Résolution :

- En optant pour un algorithme glouton, on choisit en priorité les objets ayant le prix massique le plus élevé et on essaye de remplir le sac à son maximum, ce qui donne :

	Obj1	Obj2	Obj3	Obj4
Masse (kg)	18	9	6	3
Prix (€)	60	28	16	7
Prix/Massique	3,33333333	3,11111111	2,66666667	2,33333333

CONTENU SAC A DOS 30 kg maxi					TOTAL
GLOUTON	Masse (kg)	18	9	3	30
	Prix (€)	60	28	7	95

1^{er} choix car
meilleur
prix/massique

En 2^e choix il reste 12 kg
transportable donc obj2 est le
meilleur en étant inférieur à 12 kg

En 3^e choix il reste 3 kg
transportable donc obj4
s'impose par défaut

Le Problème du Sac à Dos

- Résolution :
 - D'autres configurations étaient possibles tout en respectant la charge maxi de 30 kg, mais le prix global est inférieur à celui obtenu avec l'algorithme glouton.

	CONTENU SAC A DOS 30 kg maxi				TOTAL
Masse	9	9	9	3	30
Prix	28	28	28	7	91

ou

	CONTENU SAC A DOS 30 kg maxi			TOTAL
Masse	18	6	6	30
Prix	60	16	16	92

	Obj1	Obj2	Obj3	Obj4
Masse (kg)	18	9	6	3
Prix (€)	60	28	16	7
Prix/Massique	3,33333333	3,11111111	2,66666667	2,33333333

Le Problème du Sac à Dos

- Contre exemple de mauvaise performance de l'algorithme glouton :
 - Cependant, comme pour le rendu de monnaie, l'algorithme glouton ne garantit pas de trouver la ou l'une des solutions optimales !
 - Exemple :

	Obj1	Obj2	Obj3	Obj4
Masse (kg)	12	9	6	3
Prix (€)	40	29	6	2
Prix/Massique	3,33333333	3,22222222	1	0,66666667

Le Problème du Sac à Dos

- **Contre exemples de mauvaise performance de l'algorithme glouton :**
 - L'algorithme glouton va sélectionner les objets ayant le prix/massique le plus élevé, soit ici l'objet 1 deux fois puis l'objet 3 une fois pour atteindre les 30 kg.
 - On obtient 3 objets pour une valeur de 86 €.

		CONTENU SAC A DOS 30 kg maxi			TOTAL
GLOUTON	Masse	12	12	6	30
	Prix	40	40	6	86

Le Problème du Sac à Dos

- **Contre exemples de mauvaise performance de l'algorithme glouton :**
 - Mais il y avait une meilleure répartition ! En effet il est plus intéressant de prendre trois objets n° 2 et un objet n°4 !

		CONTENU SAC A DOS 30 kg maxi			TOTAL
GLOUTON	Masse	12	12	6	30
	Prix	40	40	6	86

		CONTENU SAC A DOS 30 kg maxi				TOTAL
OPTIMUM	Masse	9	9	9	3	30
	Prix	29	29	29	2	89

Le Problème du Sac à Dos

- **Implémentation de l'algorithme glouton pour le problème du sac à dos :**

- L'algorithme est assez proche de celui du rendu de monnaie. Les principales différences sont :
 - Le stockage des données masse et prix de chaque objet seront stockées dans une liste objet qui sera elle-même contenue dans une liste recensant l'ensemble des listes des objets.
 - Le programme calculera le rapport prix massique de chaque objet et classera les objets dans l'ordre décroissant des prix massiques.

- Exemple de liste pour la saisie des objets :

Identifiant objet	Obj1	Obj2	Obj3	Obj4
Masse (kg)	18 kg	9 kg	6 kg	3 kg
Prix (€)	60 €	28 €	16 €	7 €



```
objetsType = [[18, 60],  
               [9, 28],  
               [6, 16],  
               [3, 7]]
```

Le Problème du Sac à Dos

- **Implémentation de l'algorithme glouton pour le problème du sac à dos :**

- Votre programme en 1^{ère} partie devra calculer le rapport prix massique de chaque objet et trier les objets dans l'ordre décroissant (ou croissant selon votre algorithme).
- Nous vous fournissons ci-dessous le code qui permet de calculer le rapport prix massique et de l'ajouter dans la liste de description des objets.

```
4
5 def addPrixMassique(objets):
6     # Calcul des prix/massiques par objet, et ajout du prix massique à l'indice 0 de la liste
7     # de définition de chaque objet.
8     for i in range(len(objets)):
9         objets[i] = [objets[i][1] / objets[i][0], objets[i][0], objets[i][1]]
10
11     # Classement dans l'ordre décroissant des objets selon leur prix/massique :
12     objets.sort(reverse = True)
13
```

Le Problème du Sac à Dos

- **Implémentation de l'algorithme glouton pour le problème du sac à dos :**
 - Ensuite vous pouvez définir une 1^{ère} version de la fonction de remplissage du sac à dos :

```
15 def sacAdos_v1(objets, masseMax):
16     """Fonction appliquant l'algorithme glouton pour le problème du sac à dos
17         en mode AVEC remise des objets."""
18
19     contenuSacAdos = []
20     addPrixMassique(objets)
21     masseSac = 0
22     # Boucle de remplissage du sac :
23     i = 0
24
25
26
27
28
29
30
31
32     return contenuSacAdos
```

Code à compléter

Le Problème du Sac à Dos

- **Implémentation de l'algorithme glouton pour le problème du sac à dos :**
 - Le code principal doit ressembler à quelque chose de ce genre :

```
58 # Liste contenant les sous-listes associées à chaque objet au format :  
59 # [[masse1, prix1], [masse2, prix2], ... [masseN, prixN]]  
60 objetsTest01 = [ [12, 40],  
61                  [9, 29],  
62                  [6, 6],  
63                  [3, 2]]  
64  
65 objetsTest02 = [[18, 60],  
66                 [9, 28],  
67                 [6, 16],  
68                 [3, 7]]  
69  
70 print(sacAdos_v1(objetsTest01, 30))  
71
```

Le Problème du Sac à Dos

- **Implémentation de l'algorithme glouton pour le problème du sac à dos :**
 - L'exécution du programme permet d'obtenir ceci dans la console :

```
>>> %Run '02-Sac à Dos.py'  
[[3.3333333333333335, 12, 40], [3.3333333333333335, 12, 40], [1.0, 6, 6]]
```

Applications

Version sans remise



Le Problème du Sac à Dos

Le Problème du Sac à Dos sans remise

- **Explication :**
 - La 1^{ère} version développée est dite avec remise car on pouvait prendre les objets plusieurs fois et sans limite.
 - Nous vous demandons maintenant de proposer une seconde version de votre programme du sac à dos qui assure un remplissage sans prendre plusieurs fois le même objet.

Le Problème du Sac à Dos sans remise

- **Exemple :**

- Si on reprend notre second exemple, l'algorithme glouton pour cette configuration trouve le résultat optimal qui ici consiste à prendre tout simplement l'ensemble des objets :

	Obj1	Obj2	Obj3	Obj4
Masse (kg)	12	9	6	3
Prix (€)	40	29	6	2
Prix/Massique	3,33333333	3,22222222	1	0,66666667

		CONTENU SAC A DOS 30 kg maxi				TOTAL
GLOUTON	Masse	12	9	6	3	30
	Prix	40	29	6	2	77

Le Problème du Sac à Dos sans remise

- **Contre exemple :**

- Mais comme pour le problème avec remise, l'algorithme glouton ne va pas proposer la solution optimale dans certaines situations comme celle présentée ci-dessous :

	Obj1	Obj2	Obj3	Obj4	Obj5
Masse (kg)	24	15	12	6	3
Prix (€)	80	49	38	16	9
Prix/Massique	3,33333333	3,26666667	3,16666667	2,66666667	3

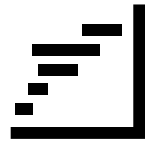
		CONTENU SAC A DOS 30 kg maxi		TOTAL
GLOUTON	Masse	24	3	27
	Prix	80	9	89

		CONTENU SAC A DOS 30 kg maxi			TOTAL
OPTIMUM	Masse	15	12	3	30
	Prix	49	38	9	96

Le Problème du Sac à Dos sans remise

- **Mise en œuvre :**
 - Modifier votre programme précédent pour en créer une nouvelle version permettant de remplir le sac à dos sans réaliser de remise du même objet.
 - Tester votre programme.
- **Remarque :** La modification consiste à modifier/supprimer que 1 à 2 de lignes de code !

Applications

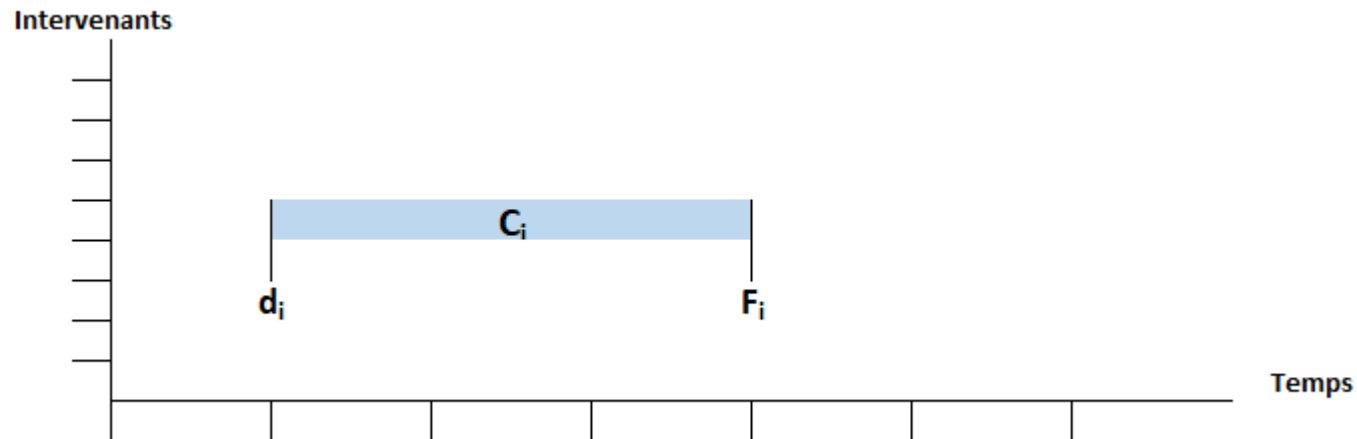


Création d'un planning

Problème de l'ordonnancement de tâches

- **Description du problème initial :**

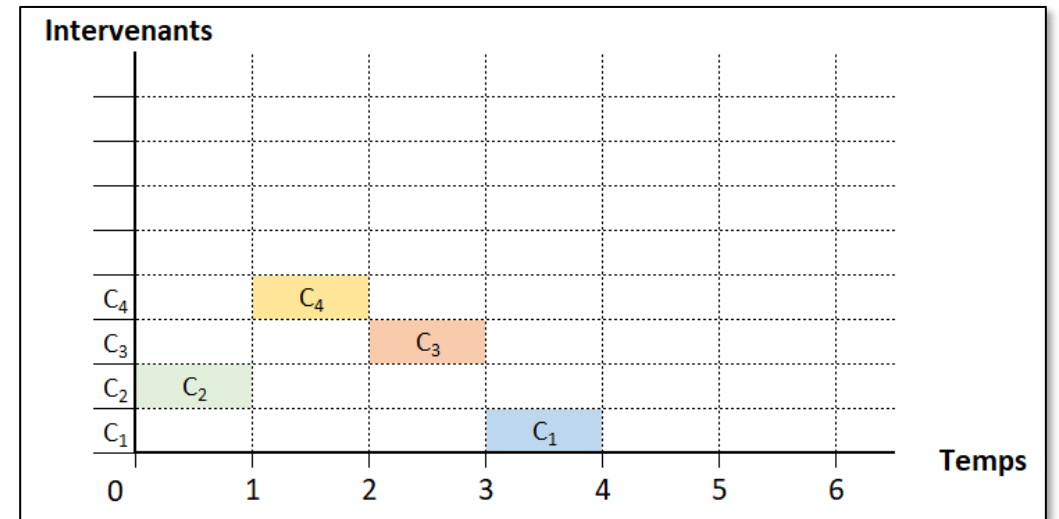
- Des conférenciers sont invités à présenter leurs exposés dans une salle. Mais leurs disponibilités ne leur permettent d'intervenir qu'à des horaires bien définis. Le problème est de construire un planning d'occupation de la salle avec le plus grand nombre de conférenciers. Désignons par n , entier naturel non nul, le nombre de conférenciers. Chacun d'eux, identifié par une lettre C_i , où i est un entier compris entre 0 et $n - 1$, est associé à un intervalle temporel $[d_i, f_i[$ où d_i et f_i désignent respectivement l'heure de début et l'heure de fin de l'intervention. Afin de dégager une tactique de résolution du problème, commençons par analyser plusieurs situations.



Problème de l'ordonnancement de tâches

- **Etude de cas 1 :**

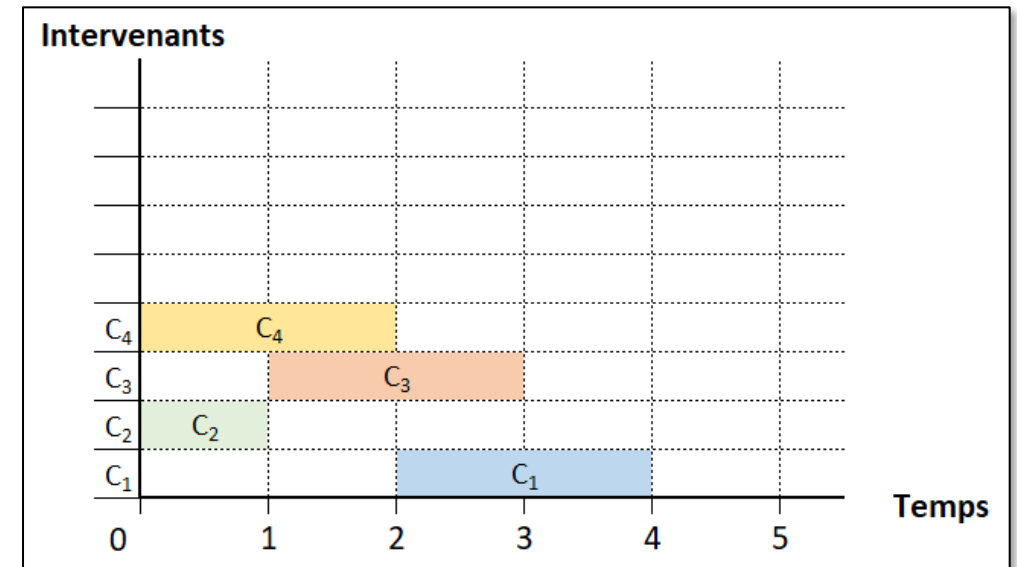
- 4 conférenciers peuvent intervenir aux intervalles temporels suivants :
 - $C_1 : [3, 4[$
 - $C_2 : [0, 1[$
 - $C_3 : [2, 3[$
 - $C_4 : [1, 2[$
- Une telle situation est simple puisque tous les conférenciers peuvent intervenir sur des créneaux horaires disjoints.
- Le planning est donc défini par la suite $[C_2, C_4, C_3, C_1]$ de conférenciers.



Problème de l'ordonnancement de tâches

- **Etude de cas 2 :**

- On reprend 4 conférenciers mais avec des créneaux pas toujours disjoints :
 - $C_1 : [2, 4[$
 - $C_2 : [0, 1[$
 - $C_3 : [1, 3[$
 - $C_4 : [0, 2[$
- Ici **il faut faire des choix** et notamment ne pas retenir toutes les conférences. Les solutions possibles sont :
- $[C_2, C_1]$ ou $[C_2, C_3]$ ou $[C_4, C_1]$ ou $[C_3]$



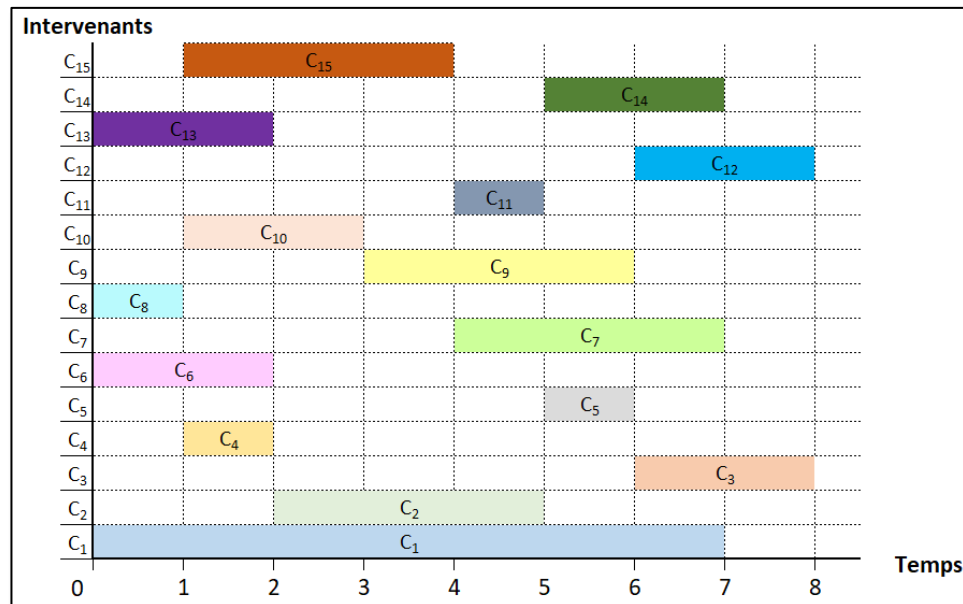
La stratégie gloutonne va nous permettre d'appliquer un choix !

Problème de l'ordonnancement de tâches

- Proposition & description d'un algorithme selon une stratégie glouton :
 - Classer les intervalles par heures de fin f_i croissantes ;
 - Choisir le conférencier C_i associé au 1^{er} intervalle issu du tri croissant selon f_i
 - Choisir parmi les intervalles suivant du classement le 1^{er} qui est compatible avec le 1^{er} conférencier;
 - Recommencer avec les intervalles classés suivant jusqu'à que l'ensemble des intervalles soient traités.

Problème de l'ordonnancement de tâches

- **Proposition & description d'un algorithme selon une stratégie glouton :**
 - Avant de se lancer dans la rédaction du code, il faut mener une réflexion sur la façon de représenter les intervalles des conférenciers;
 - On désire saisir les 15 intervalles suivants :



Problème de l'ordonnancement de tâches

- **Proposition & description d'un algorithme selon une stratégie glouton :**
 - On va construire une liste, cette liste comporte des sous listes représentant l'intervalle sous le format :
 - $[d_i, f_i, "C_i"]$

```
intervallesConf = [[0,7,'C1'],[2,5,'C2'],[6,8,'C3'],  
                  [1,2,'C4'],[5,6,'C5'],[0,2,'C6'],  
                  [4,7,'C7'],[0,1,'C8'],[3,6,'C9'],  
                  [1,3,'C10'],[4,5,'C11'],[6,8,'C12'],  
                  [0,2,'C13'],[5,7,'C14'],[1,4,'C15']]
```

Problème de l'ordonnancement de tâches

- **Proposition & description d'un algorithme selon une stratégie glouton :**

- Afin de vous aider dans votre tâche, on vous donne la trame du programme et les instructions les plus délicates comme celles qui assurent le tri initial.

```
3
4 def organiserPlanning(intervalles):
5     planning = []
6     # Tri des intervalles dans l'ordre des dates de fin croissantes :
7     intervalles.sort(key = lambda intervalle:intervalle[1])
8
9     # Construction du planning selon une sélection gloutonne :
10    i = 0
11    j = i + 1
12    planning.append(intervalles[i])
13    while Code à compléter :
14        # Vérification de la compatibilité des intervalles :
15        if Code à compléter :
16            planning.append(intervalles[j])
17            i = j # On poursuit le travail sur le nouveau intervalle ajouté au planning.
18        else :
19            j = j + 1
20    return planning
21
```

Problème de l'ordonnancement de tâches

- Suite avec une proposition de fonction d'affichage du planning :

```
22
23 def afficherPlanning(listePlanning):
24     t = ""
25     for d, f, l in listePlanning:
26         t += l + "\t:" + " " * d + "#" * (f-d) + "\n"
27     print(t)
28
```

Problème de l'ordonnancement de tâches

- Et la partie du programme principal :

```
30
31 # Tests et exemples :
32 # -----
33
34 intervallesConf = [[0,7,'C1'],[2,5,'C2'],[6,8,'C3'],
35                   [1,2,'C4'],[5,6,'C5'],[0,2,'C6'],
36                   [4,7,'C7'],[0,1,'C8'],[3,6,'C9'],
37                   [1,3,'C10'],[4,5,'C11'],[6,8,'C12'],
38                   [0,2,'C13'],[5,7,'C14'],[1,4,'C15']]
39
40 listePlanning = organiserPlanning(intervallesConf)
41 print(listePlanning)
42 afficherPlanning(listePlanning)
43
```

Problème de l'ordonnancement de tâches

- Le résultat :

```
>>> %Run 03-Planning.py
[[0, 1, 'C8'], [1, 2, 'C4'], [2, 5, 'C2'], [5, 6, 'C5'], [6, 8, 'C3']]
C8      :#
C4      : #
C2      :  ###
C5      :    #
C3      :      ##
>>>
```

Fin